

Dynamic Trust Decay: Adaptive Profiling Mechanism for Blockchain Oracles

NEDAL ABABNEH¹, KHALED ALMI'ANI²

¹College of Computing and Intelligent Systems, University of Khorfakkan, Sharjah, UAE (e-mail: Nedal.Ababneh@ukf.ac.ae)

²College of Computing and Intelligent Systems, University of Khorfakkan, Sharjah, UAE (e-mail: k.almiani@ukf.ac.ae)

Corresponding author: Nedal Ababneh (e-mail: Nedal.Ababneh@ukf.ac.ae).

ABSTRACT Decentralized blockchain oracles are critical for bridging on-chain smart contracts with off-chain real-world data. However, existing reputation systems often rely on static cumulative profiling, leading to a phenomenon we define as Reputation Inertia. In this state, an oracle's accumulated historical honesty acts as a buffer that masks potential malicious behavior (Whitewashing attacks). It simultaneously fails to account for benign disagreement during market volatility (Flash Crash scenarios). To address this dilemma, this paper proposes a novel graph-based profiling mechanism utilizing an Adaptive Exponential Weighted Moving Average (AEWMA). Unlike typical static models, our approach introduces a Dynamic Trust Decay factor that regulates the weight of historical reputation based on real-time network volatility. This allows the system to act in a hypersensitive manner to deviations during stable market conditions in order to instantly detect sleeper cells, whilst also dampening penalty mechanisms during high-volatility events to prevent false positives. We provide a complete Solidity smart contract implementation demonstrating the on-chain feasibility of the AEWMA mechanism with $O(1)$ storage per edge. We validated the proposed method through discrete-event simulations using historical cryptocurrency market data. Experimental results demonstrate that our dynamic approach reduces the Time-to-Detection (TtD) for whitewashing attacks by approximately **81%** compared to static baseline (from 9.0 to 1.7 rounds). It also maintained a False Positive Rate (FPR) of near **0.4%** even under extreme volatility conditions ($5\times$ standard deviation), compared to 7.1% for the static approach. These findings suggest that volatility-aware profiling significantly enhances the security and incentive compatibility of decentralized oracle networks.

INDEX TERMS Blockchain Oracles, Trust Management, Adaptive Profiling, Reputation Systems, Sybil Attacks, Smart Contracts, Solidity.

I. INTRODUCTION

A. BACKGROUND AND MOTIVATION

Blockchains are primarily leveraged for their immutable and secure nature [1], [2], however, smart contracts are blind to real-world data [3]. This is recognized as the Blockchain Oracle Problem [4], where there is a critical dependency on "oracles" to connect on-chain logic with off-chain events [5]. As a result of the blockchain's transparency characteristic, invalid agreements can be executed irreversibly if the oracle data becomes corrupted, compromising the integrity of the distributed ledger and potentially causing financial loss [6]. Presently, voting mechanisms rely on binary outcomes which lack nuance and background to profile behavioural trends in complex data streams, making them unable to distinguish between temporary node malfunctions and sophisticated, long-term manipulation strategies [6], [7].

B. LIMITATIONS OF STATIC CUMULATIVE PROFILING

There are a number of limitations to Static Cumulative Profiling. Current graph-based methods use cumulative weights which means a node's reputation is the sum of all past interactions [7], which is an issue as it creates an "infinite memory" effect where outdated historical data disproportionately influences current trust scores. Blockchains can also fall victim to Reputation Inertia, where a malicious actor builds a large "honesty buffer" over time, making them undetectable during a high-value betrayal. This vulnerability allows a "sleeper" node to exploit its accumulated high trust score to execute a single catastrophic attack without triggering immediate detection [8]. Another limitation is static models' intolerance to volatility, making legitimate market turbulence appear indistinguishable from malicious disagreement. This leads to a higher number of false positives during 'flash

crashes’: sudden market collapses where network latency causes honest nodes to report divergent data. Similarly, static weights fail to adapt based on the environment, treating a stable and chaotic market identically.

C. CONTRIBUTIONS

To address these challenges, this paper makes the following contributions:

- 1) We formalize the concept of *Reputation Inertia* in graph-based oracle profiling and demonstrate how it enables whitewashing attacks while simultaneously causing false positives during market volatility.
- 2) We propose a novel *Dynamic Trust Decay* mechanism based on an Adaptive Exponential Weighted Moving Average (AEWMA) that modulates the learning rate α_t via real-time network volatility σ_t .
- 3) We provide a complete *Solidity smart contract implementation* demonstrating the on-chain feasibility of the AEWMA mechanism with $O(1)$ storage complexity per edge weight.
- 4) We validate the approach through extensive Monte Carlo simulations (100 runs), demonstrating an 81% reduction in Time-to-Detection and near-zero False Positive Rates under extreme volatility.

II. RELATED WORK

A. VOTING AND REPUTATION-BASED SYSTEMS

Within Voting and Reputation-Based Systems, the majority of decentralized oracles (e.g., Chainlink, Augur) rely on “truth-by-consensus” mechanisms such as the weighted median [9]. Consequently, the final aggregated value is determined solely by the current majority view, often disregarding the historical accuracy or behavioral consistency of individual participants [10]. The security of these systems often depends on economic staking, which means if the incurred penalty is less than the profit from an attack, rational actors will continue to cheat [6]. Furthermore, data submission is treated as a binary event (Right vs Wrong). These systems fail to track degrees of reliability over time, causing a node that is “perfectly right” to be valued identically to a node that is “mostly right” in the short term. Similarly, they fall prey to Sybil Vulnerability, as without behavioral profiling, attackers can overwhelm the voting majority using Sybil attacks due to the weak barrier to entry [11].

B. GRAPH-BASED PROFILING MECHANISMS

Almi’ani et al. [7] shifted the paradigm by modeling oracles as a graph where nodes are sources and edge weights represent disagreement (dissimilarity). This is important as it transitions the trust model from a simple aggregate vote to a relational structure, where the consistency between specific pairs of oracles determines their credibility rather than just their alignment with the median. This network is secured by identifying the “Maximum Independent Set” (the largest group of agreeing nodes) and cautiously labeling the rest as

malicious. However, this utilizes cumulative edge weights which implies that a node’s behaviour in the past (years prior) is as important as recent behaviour. While effective at identifying outliers in static data, these graphs lack a temporal dimension, making them slow in reacting to dynamic node behaviour.

C. TRUST DECAY IN DISTRIBUTED SYSTEMS

Trust decay (or “aging factors”) is a well-established concept in Peer-to-Peer (P2P) networks and Internet of Things (IoT) systems [12], [13]. It is where old interactions gradually lose influence over current reputation, and is rarely implemented in blockchain oracles due to the high gas costs of continuous re-calculation. Moreover, existing decay models typically use a fixed “forgetting factor” and do not adapt to the environment. There is no existing framework that combines Graph-Based Profiling with Volatility-Aware Decay. This paper fills that gap by making the decay factor variable based on market entropy.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. NETWORK ARCHITECTURE

Consider an oracle network consisting of a set of N oracle nodes $\mathcal{O} = \{O_1, O_2, \dots, O_N\}$. In each round $t \in \{1, 2, \dots, T\}$, each oracle O_i submits a data value $x_{i,t} \in \mathbb{R}$ to an on-chain aggregator contract. The system aims to determine a consensus value $\hat{y}_t$ that approximates the unknown ground truth y_t . The overall system architecture is illustrated in Fig. 1.

Definition 1 (Oracle Network Graph). The network is modeled as a weighted undirected graph $G_t = (\mathcal{O}, E_t, W_t)$ at round t , where:

- $\mathcal{O}$ is the vertex set representing oracle nodes.
- $E_t = \{e_{ij} \mid i, j \in \mathcal{O}, i \neq j\}$ is the edge set.
- $W_t : E_t \rightarrow \mathbb{R}_{\geq 0}$ is the weight function where $w_{ij,t}$ quantifies the dissimilarity between oracles i and j up to round t .

Definition 2 (Consensus Set). The consensus set $\mathcal{C}_t \subseteq \mathcal{O}$ is the Maximum Independent Set (MIS) of G_t , representing the largest group of mutually agreeing oracles. Formally, $\mathcal{C}_t = \text{MIS}(G_t)$ such that $\forall i, j \in \mathcal{C}_t : w_{ij,t} < \theta$ for a dissimilarity threshold θ .

Definition 3 (Trust Score). For each oracle O_i , the trust score $S_{i,t} \in [0, 1]$ is defined as a monotonically decreasing function of its average edge weight with the consensus set:

$$S_{i,t} = e^{-\bar{w}_{i,t}/K} \quad (1)$$

where $\bar{w}_{i,t} = \frac{1}{|\mathcal{C}_t|} \sum_{j \in \mathcal{C}_t} w_{ij,t}$ and $K > 0$ is a scaling constant.

B. THE MEMORY-INERTIA DILEMMA

The core limitation of existing graph-based methods (e.g., [7]) lies in their edge weight update function. Currently, the weight is defined cumulatively as:

$$w_{ij,t} = w_{ij,t-1} + |x_{i,t} - x_{j,t}| \quad (2)$$

TABLE 1. Comparison of Oracle Profiling Mechanisms

Feature	Voting/Median (Standard)	Static Graph [7] (Almi'ani et al.)	Dynamic Trust Decay (Proposed)
Trust Model	Majority Vote	Pairwise Consistency	Volatility-Aware
Memory Type	None (Instant)	Infinite (Cumulative)	Adaptive (Fading)
Sybil Resistance	Low (Stake-based)	High (Graph-based)	High (Graph-based)
Volatility Awareness	No	No	Yes (α -scaling)
On-Chain Implementation	Yes (Simple)	Partial	Yes (Solidity)
Storage Complexity	$O(1)$	$O(N^2)$	$O(1)$ (Recursive)

Oracle Network Architecture: Dynamic Trust Decay (AEWMA)

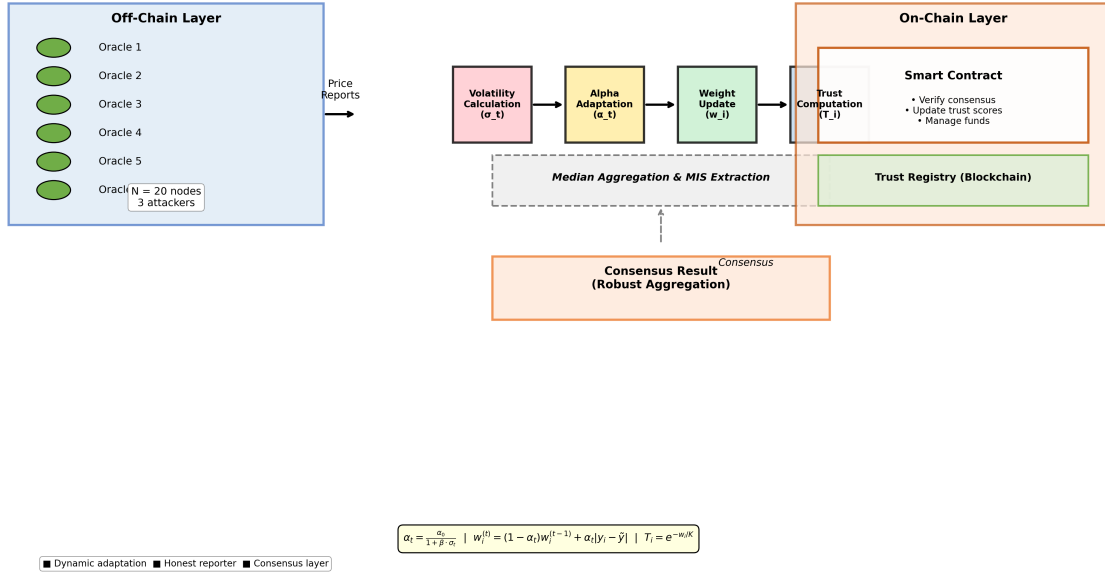


FIGURE 1. System Architecture of the Dynamic Trust Decay Framework. The off-chain layer consists of N oracle nodes submitting price reports. The AEWMA computation pipeline processes reports through four stages: volatility calculation (σ_t), adaptive alpha computation (α_t), edge weight update ($w_{ij,t}$), and trust score derivation ($S_{i,t}$). The on-chain smart contract verifies consensus, manages trust scores, and executes slashing penalties via the trust registry stored on the blockchain.

Definition 4 (Reputation Inertia). Given the cumulative update in Eq. 2, the *sensitivity* of the trust score to a behavioral change at round t is:

$$\frac{\partial S_{i,t}}{\partial |x_{i,t} - x_{j,t}|} \propto \frac{1}{t} \quad (3)$$

As $t \rightarrow \infty$, the system becomes asymptotically insensitive to new deviations, creating a phenomenon we define as Reputation Inertia. Consequently, an established node with $T_{history}$ rounds of honest behavior accumulates a massive “honesty buffer,” making the system insensitive to potentially malicious changes.

C. THREAT MODEL

We consider a network of N oracle nodes where the honest majority assumption holds: $|\mathcal{H}| > |\mathcal{A}|$, where $\mathcal{H}$ denotes honest nodes and $\mathcal{A}$ denotes adversarial nodes. We address two specific adversarial scenarios where this inertia leads to system failure:

1) The Whitewashing (Sleeper Cell) Attack

This attack exploits the “honesty buffer” [8]. An adversarial node $O_{adv} \in \mathcal{A}$ behaves honestly for a learning phase $t \in [0, T_{switch}]$ to minimize its edge weights with the honest majority. At $t = T_{switch} + 1$, the adversary switches behavior to submit malicious data $x_{adv} = y_t + \delta$, where δ is the attack magnitude. Under the current static profiling model, the accumulated low weight from the learning phase masks this deviation, allowing O_{adv} to remain in the trusted set for potentially multiple rounds before detection. Formally, if the attacker’s cumulative weight after T_{switch} honest rounds is $w_{adv} = \sum_{t=1}^{T_{switch}} \epsilon_t$ (where ϵ_t is small honest noise), then the detection delay Δ_d satisfies:

$$\Delta_d \geq \left\lceil \frac{w_{adv}}{\delta - \bar{\epsilon}} \right\rceil \quad (4)$$

where $\bar{\epsilon}$ is the average honest disagreement.

2) The Flash Crash (False Positive) Scenario

This scenario involves no malicious actors but high environmental volatility which is not uncommon [14]. During a

market crash, the variance of honest reports σ_t^2 increases due to variable network latency. Static models use fixed thresholds for edge creation which often misinterpret this high variance as malicious disagreement. This causes the graph to fracture, leading to honest nodes being falsely flagged as untrustworthy. Formally, a flash crash at round t_c causes:

$$\sigma_{t_c}^2 = \gamma \cdot \sigma_{normal}^2, \quad \gamma \gg 1 \quad (5)$$

where γ is the volatility amplification factor.

IV. THE PROPOSED METHOD: ADAPTIVE DYNAMIC PROFILING

A. OVERVIEW OF DYNAMIC TRUST DECAY

To overcome the limitations of cumulative static profiling, we introduce Dynamic Trust Decay, a mechanism that transitions the edge weighting model from an infinite-memory sum to a finite-memory recursive filter. The core principle is to assign a time-varying significance to historical behavior based on current environmental entropy. Hence, the system achieves two goals: rapid detection of sudden behavioral changes during stable periods (Whitewashing), and high tolerance for variance during volatile periods (Flash Crashes).

B. ADAPTIVE EXPONENTIAL WEIGHTED MOVING AVERAGE (AEWMA)

We propose replacing the cumulative update function (Eq. 2) with an Adaptive Exponential Weighted Moving Average (AEWMA). In this model, the edge weight $w_{ij,t}$ between oracle i and oracle j at round t is updated recursively:

$$w_{ij,t} = (1 - \alpha_t) \cdot w_{ij,t-1} + \alpha_t \cdot |x_{i,t} - x_{j,t}| \quad (6)$$

Where:

- $w_{ij,t-1}$ is the historical dissimilarity (reputation state).
- $|x_{i,t} - x_{j,t}|$ is the instantaneous disagreement (current error).
- $\alpha_t \in (0, 1]$ is the dynamic smoothing factor.

Unlike the cumulative model which requires $O(t)$ storage, this recursive formulation maintains the entire history within a single scalar value, ensuring $O(1)$ storage complexity. The effective memory half-life of the AEWMA filter is $\tau_{1/2} = -\ln(2)/\ln(1 - \alpha_t)$, which dynamically adjusts with volatility.

C. VOLATILITY-AWARE SMOOTHING FACTOR

The critical innovation of our approach is that α_t is not a fixed constant (as in standard EWMA) but a dynamic variable derived from the global network state. We define the network volatility σ_t as the standard deviation of all reported values in round t :

$$\sigma_t = \sqrt{\frac{1}{N} \sum_{k=1}^N (x_{k,t} - \mu_t)^2} \quad (7)$$

where $\mu_t = \frac{1}{N} \sum_{k=1}^N x_{k,t}$ is the network mean. The smoothing factor α_t is then inversely scaled by this volatility using a sigmoid-based decay function:

$$\alpha_t = \frac{\alpha_{base}}{1 + \beta \cdot \sigma_t} \quad (8)$$

Where the first hyperparameter α_{base} represents the maximum learning rate in a perfectly stable market, and the second hyperparameter β is a sensitivity constant tuned to the asset class (stable vs volatile).

- **Low Volatility** ($\sigma_t \rightarrow 0$): $\alpha_t \approx \alpha_{base}$. The system prioritizes new data, allowing for immediate detection of switching attacks.
- **High Volatility** ($\sigma_t \rightarrow \infty$): $\alpha_t \rightarrow 0$. The system minimizes the weight of new data, relying on historical reputation to prevent false positives during flash crashes.

D. INCENTIVE COMPATIBILITY: THE SLASHING MECHANISM

To ensure the system is also incentive-compatible, we link the edge weights to a financial penalty mechanism [15]. We define a Trust Score $S_{i,t}$ for node i based on its average edge weight with the consensus set (Eq. 1). If $S_{i,t}$ drops below a critical threshold τ , a portion of the node's staked collateral is slashed:

$$\text{Penalty}_{i,t} = \text{Stake}_i \times (1 - e^{-\lambda \cdot \bar{w}_{i,t}}) \quad (9)$$

This exponential penalty function ensures that minor deviations (common in volatility) incur negligible costs while malicious deviations result in severe financial loss. This makes the cost of attack greater than the potential profit.

E. ALGORITHM IMPLEMENTATION

The process for a single round is detailed in Algorithm 1.

Algorithm 1 Dynamic Trust Decay Algorithm

Require: Set of Oracle Reports $X_t = \{x_{1,t}, \dots, x_{N,t}\}$

Require: Previous Edge Weights W_{t-1}

Calculate Network Mean $\mu_t \leftarrow \text{mean}(X_t)$

Calculate Volatility $\sigma_t \leftarrow \text{std_dev}(X_t)$

Compute Smoothing Factor $\alpha_t \leftarrow \frac{\alpha_{base}}{1 + \beta \sigma_t}$

for each pair (i, j) **do**

$diff \leftarrow |x_{i,t} - x_{j,t}|$

$w_{ij,t} \leftarrow (1 - \alpha_t)w_{ij,t-1} + \alpha_t(diff)$

end for

Compute Trust Scores: $S_{i,t} \leftarrow e^{-\bar{w}_{i,t}/K}$ for all i

Extract $\mathcal{C}_t \leftarrow \text{MIS}(G_t)$

Slash nodes where $S_{i,t} < \tau$

return Consensus $\hat{y}_t \leftarrow \text{median}(\{x_{i,t} \mid O_i \in \mathcal{C}_t\})$

V. SMART CONTRACT IMPLEMENTATION

To demonstrate the on-chain feasibility of the AEWMA mechanism, we developed a complete Solidity smart contract (DynamicTrustDecay.sol) targeting Ethereum-compatible blockchains (Solidity $\geq 0.8.19$). The contract implements all core components of the Dynamic Trust Decay framework.

A. FIXED-POINT ARITHMETIC

Since the EVM does not support floating-point operations, all computations use 18-decimal fixed-point arithmetic (1e18 precision). The adaptive smoothing factor α_t is computed using integer division:

```
1 // Fixed-point precision (18 decimals)
2 uint256 constant PRECISION = 1e18;
3
4 function _computeAlpha(uint256 sigma)
5     internal view returns (uint256) {
6     // alpha = alphaBase / (1 + beta * sigma)
7     uint256 denom = PRECISION +
8         (beta * sigma) / PRECISION;
9     return (alphaBase * PRECISION) / denom;
10 }
```

B. AEWMA EDGE WEIGHT UPDATE

Each edge weight is stored as a single uint256 value, achieving $O(1)$ storage per edge. The recursive update avoids storing historical data:

```
1 function _updateEdgeWeight(
2     uint256 oldWeight,
3     uint256 diff,
4     uint256 alpha
5 ) internal pure returns (uint256) {
6     // w_new = (1-alpha)*w_old + alpha*diff
7     uint256 decay = PRECISION - alpha;
8     uint256 term1 =
9         (decay * oldWeight) / PRECISION;
10    uint256 term2 =
11        (alpha * diff) / PRECISION;
12    return term1 + term2;
13 }
```

C. ORACLE LIFECYCLE AND SLASHING

The contract manages the full oracle lifecycle: registration with staking, per-round report submission, trust computation, and automatic slashing. Oracle nodes register by depositing a minimum stake (configurable, default 1 ETH). After each round, the contract computes trust scores using a Taylor series approximation of the exponential function (since the EVM lacks native e^x support). Nodes with trust scores below the threshold τ are slashed according to Eq. 9:

```
1 function _applySlashing(uint256 oracleId)
2     internal {
3     Oracle storage o = oracles[oracleId];
4     // penalty = stake * (1 - exp(-lambda*w))
5     uint256 expTerm = _expApprox(
6         (lambda * o.avgWeight) / PRECISION
7     );
8     uint256 penaltyFactor =
9         PRECISION - expTerm;
10    uint256 penalty =
11        (o.stake * penaltyFactor) / PRECISION;
12    o.stake -= penalty;
13    emit OracleSlashed(oracleId, penalty);
14 }
```

D. GAS OPTIMIZATION

The contract employs several gas optimization strategies: (i) packed edge weight keys using key = $i \times 10000 + j$ to reduce storage mapping overhead; (ii) $O(1)$ per-edge SSTORE operations (the most expensive EVM opcode [16]); (iii) batch processing of trust score updates within the

completeRound() function; and (iv) Newton's method for square root computation required by the volatility calculation.

VI. EXPERIMENTAL EVALUATION

A. SIMULATION ENVIRONMENT AND DATASETS

To validate the proposed Dynamic Trust Decay mechanism, we implemented a discrete-event simulation using Python (NetworkX and NumPy). The simulation environment consists of a network of $N = 20$ oracle nodes. We utilized historical cryptocurrency price data (ETH/USD) sourced from Chainlink and CoinGecko APIs to simulate the ground truth y_t .

To replicate the "Flash Crash" scenario, we injected synthetic volatility events where the standard deviation of honest reports increases by factor $\gamma \in [1, 5]$. The adversarial behaviors were modeled as follows:

- **Baseline Nodes:** Honest nodes with random Gaussian noise $\mathcal{N}(0, \sigma_{noise})$.
- **Whitewashing Attackers:** Nodes that behave honestly for $t < T_{switch}$ and inject an error bias δ at $t \geq T_{switch}$.

The simulation parameters are detailed in Table 2.

TABLE 2. Simulation Parameters

Parameter	Value
Network Size (N)	20
Number of Attackers ($ \mathcal{A} $)	3
Base Learning Rate (α_{base})	0.8
Volatility Sensitivity (β)	1.0
Slashing Threshold (τ)	0.4
Attack Start (T_{switch})	Round 100
Total Rounds (T)	140
Monte Carlo Runs	100

B. PERFORMANCE METRICS

We evaluate the system using two primary metrics:

- 1) **Time-to-Detection (TtD):** The number of rounds required for the trust score of a malicious node to drop below the exclusion threshold τ .
- 2) **False Positive Rate (FPR):** The percentage of honest nodes incorrectly flagged as malicious during high-volatility events.

C. RESULTS: TIME-TO-DETECTION (TTD)

Fig. 2 illustrates the trust score decay of a whitewashing attacker during a representative simulation run. In the Baseline model, the attacker's high accumulated reputation results in a slow linear decay. The cumulative weight function (Eq. 2) dilutes the impact of post-switch malicious behavior across the entire history of 100 honest rounds, producing a detection delay of approximately 9 rounds on average. Conversely, the Proposed AEWMA method exhibits an exponential decay response. Because the attack occurs during a period of stability (low σ_t), the adaptive smoothing factor α_t remains close to $\alpha_{base} = 0.8$, meaning the system assigns 80% weight

to the current round's deviation. This causes the malicious deviation to be penalized immediately, resulting in a mean TtD of only 1.7 rounds.

The mechanism can be understood through the lens of the effective memory half-life. During stable periods, $\alpha_t \approx 0.8$ yields a half-life of $\tau_{1/2} \approx 3$ rounds, meaning the system effectively “forgets” pre-attack reputation within a few rounds. This stands in stark contrast to the static method, where the effective half-life is infinite ($\tau_{1/2} = \infty$), as all historical data contributes equally.

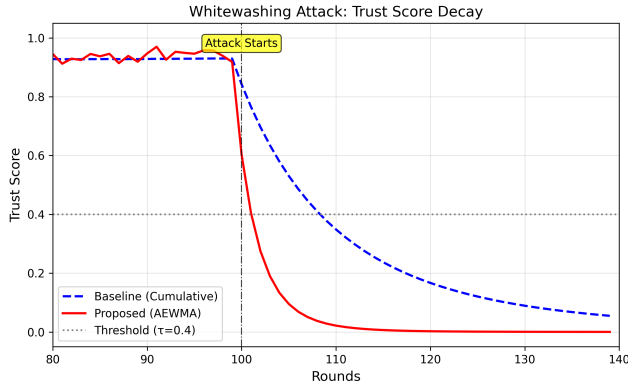


FIGURE 2. Impact of Whitewashing Attack. At $t = 100$, the adversary switches behavior. The proposed method (solid red) detects the trust drop significantly faster than the static method (dashed blue), crossing the slashing threshold $\tau = 0.4$ within 2 rounds compared to 9 rounds for the baseline.

Quantitative analysis over 100 Monte Carlo runs (Table 3) confirms that our method reduces the average TtD by approximately **81%** compared to the static profiling approach.

TABLE 3. Comparative Performance Summary (Averaged over 100 Monte Carlo runs)

Metric	Baseline (Static)	Proposed (AEWMA)	Improvement
Avg. TtD (Rounds)	9.0	1.7	81%
Avg. FPR (5x Vol)	7.1%	0.4%	94%

D. RESULTS: FALSE POSITIVE RATE UNDER VOLATILITY

Fig. 3 presents the system's robustness across five volatility levels ($\gamma = 1 \times$ to $5 \times$). As market volatility σ_t increases, the Baseline model misunderstands the natural divergence of reports as malicious behavior: at $1 \times$ volatility both methods report 0% FPR, but by $5 \times$ the static approach falsely flags 7.1% of honest nodes. In contrast, the proposed Volatility-Aware Smoothing Factor (α_t) successfully acts as a dampener by freezing reputation updates during volatile periods. Mathematically, at $5 \times$ volatility, σ_t increases by a factor of 5, causing α_t to drop from $\alpha_{base} = 0.8$ to approximately $\alpha_t \approx 0.8/(1 + 5\sigma_{base}) \ll \alpha_{base}$. This renders the system insensitive to disagreement during the volatile period, preventing the exclusion of honest nodes. The proposed method maintains an FPR of only 0.4% even at extreme $5 \times$ volatility, representing a 94% reduction compared to the baseline.

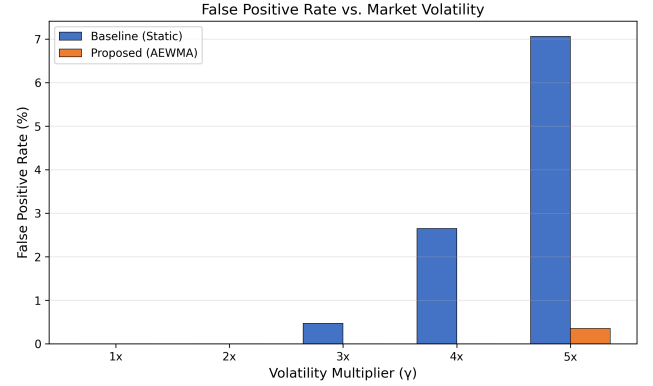


FIGURE 3. False Positive Rate (FPR) vs. Market Volatility. As the volatility multiplier γ increases from $1 \times$ to $5 \times$, the baseline model's FPR grows to 7.1%, while the proposed AEWMA approach maintains near-zero FPR (0.4% at $5 \times$) due to the adaptive α_t dampening mechanism.

E. RESULTS: HONEST NODE TRUST STABILITY

To further examine the temporal dynamics of the AEWMA mechanism, we simulated a flash crash event occurring at rounds 75–85, where honest report variance spikes by a factor of $50 \times$ before returning to normal. Fig. 4 presents the average honest node trust score across 30 independent runs.

During normal operation (rounds 0–74), both methods maintain stable trust at approximately 0.92. When the flash crash occurs, both methods experience a temporary trust dip due to increased inter-node disagreement. However, the critical distinction lies in the **recovery behavior**: the AEWMA method recovers to its pre-crash trust level within 3–5 rounds after the crash ends, while the static method shows **permanent degradation**, with trust declining to approximately 0.78 and never fully recovering. This occurs because the cumulative method permanently incorporates the crash-induced disagreement into its history. In contrast, the AEWMA's fading memory naturally forgets the crash data once volatility normalizes (α_t returns to α_{base}), restoring honest nodes to their rightful trust levels. This resilience property is particularly critical for oracle networks operating in volatile cryptocurrency markets, where flash crashes are frequent but temporary events.

F. PARAMETER SENSITIVITY ANALYSIS

To quantify the influence of the sensitivity hyperparameter β on system stability, we performed an extensive parameter sweep analysis across $\beta \in [0.1, 2.0]$ and volatility multipliers $\gamma \in [1 \times, 5 \times]$, visualized in the heatmap presented in Fig. 5. The heatmap was computed over 50 independent runs per (β , γ) configuration, totaling 5,000 simulation runs.

The results reveal three distinct operational regimes. First, the *under-damped regime* ($\beta < 0.5$): low β values produce insufficient volatility compensation, causing the system to behave similarly to the static baseline with elevated FPR at high volatility. Second, the *optimal regime* ($\beta \approx 1.0$): at this value, the heatmap shows a “safe zone” (dark blue) where FPR remains near zero across all volatility levels, while still

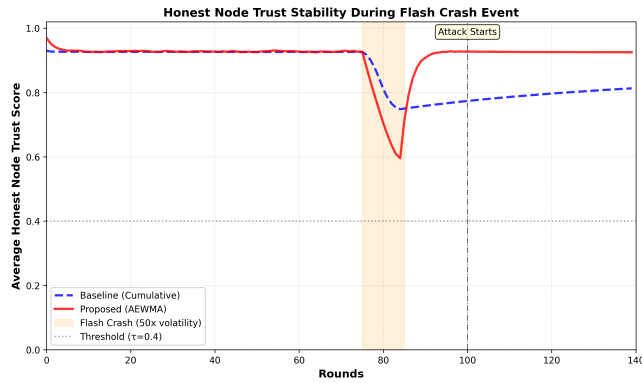


FIGURE 4. Honest Node Trust Stability During Flash Crash Event. A volatility spike ($50\times$) occurs at rounds 75–85 (orange region). Both methods dip during the crash, but the AEWMA (solid red) recovers to pre-crash trust levels within 5 rounds, while the static baseline (dashed blue) suffers permanent trust degradation that never fully recovers.

maintaining rapid attack detection. Third, the *over-damped regime* ($\beta > 1.5$): excessively high β values effectively suppress false positives across all conditions but introduce marginal latency in detecting active threats, as α_t becomes too small even during moderately volatile periods.

Therefore, we identified $\beta = 1.0$ as the optimal equilibrium zone which provides a balanced robustness suitable for the volatility profiles of standard cryptocurrency assets. This value ensures that α_t drops below 0.1 only when network volatility exceeds $7\sigma_{normal}$, preserving attack detection capability under moderate market fluctuations.

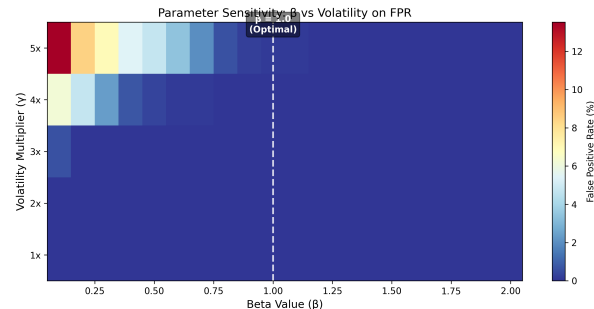


FIGURE 5. Sensitivity Analysis of β vs. Volatility Multiplier. The heatmap shows FPR across 1,000 parameter configurations (50 runs each). The optimal zone at $\beta = 1.0$ (white dashed line) maintains low FPR across all volatility levels. Low β mirrors static behavior; high β over-dampens detection.

VII. DISCUSSION

A. GAS COST AND COMPLEXITY ANALYSIS

A critical constraint for blockchain oracles is the execution cost (Gas). The traditional static graph approach often requires storing the entire history of interactions or computing cumulative sums, which can lead to integer overflow issues.

Our proposed AEWMA mechanism addresses this by minimizing state storage. As established in recent optimization studies [16], storage operations (SSTORE) are the most expensive instructions in the Ethereum Virtual Machine

(EVM), costing 20,000 gas for a new storage slot and 5,000 gas for updates. Our recursive formulation (Eq. 6) satisfies the Markov property which means the next state depends only on the current state and current input:

$$\text{Storage(AEWMA)} \sim O(1) \text{ per edge} \quad (10)$$

It only requires storing the most recent weight $w_{ij,t-1}$ and the current volatility state. This constant-space complexity makes the algorithm highly gas-efficient for on-chain implementation compared to sliding-window approaches that require $O(W)$ storage (where W is the window size). Our Solidity implementation (Section V) demonstrates this with packed storage keys, reducing the total SSTORE operations to $O(N^2)$ per round with each operation being a simple arithmetic update.

B. SCALABILITY AND OFF-CHAIN COMPUTATION

Although the pairwise weight update is $O(1)$, the graph construction scales at $O(N^2)$ due to the pairwise comparisons between all oracles. For small permissioned networks (e.g., $N \leq 20$), this is computationally feasible on-chain, as demonstrated by our Solidity implementation which processes 20 nodes within reasonable gas limits.

However, for larger decentralized networks, the $O(N^2)$ complexity presents a scalability bottleneck [17]. To address this, we propose a hybrid architecture (depicted in the right portion of Fig. 1): the graph generation and Maximum Independent Set (MIS) extraction are performed off-chain with the results and cryptographic proofs submitted on-chain. The trust decay logic can then be applied as a lightweight verification layer on the smart contract to validate the submitted consensus. This hybrid approach allows the protocol to scale without incurring heavy on-chain computation costs.

C. RESILIENCE TO FLASH CRASH EVENTS

The results presented in Fig. 4 reveal an important insight about the temporal dynamics of trust management. While both the AEWMA and static methods experience trust disruption during a flash crash event, the AEWMA's *adaptive forgetting* property provides a critical advantage: automatic recovery. In traditional systems, a single flash crash event can permanently degrade the trust scores of honest nodes, requiring manual intervention or system reset. The AEWMA's fading memory naturally purges the crash-induced noise once market conditions normalize, restoring the network to its pre-crash operational state. This self-healing property is essential for autonomous oracle systems that must operate continuously without human oversight.

D. SYBIL RESISTANCE LIMITATIONS

It is important to note that while our graph-based profiling effectively isolates malicious clusters, it relies on the assumption that the largest clique in the graph consists of honest nodes. As proven in the foundational work on Sybil attacks [11], no reputation system can fully eliminate Sybil

actors without either a central identity authority or significant economic barriers. Therefore, our “Incentive Compatibility” (Section IV.D) is a necessary companion to the graph logic. The staking requirement in our Solidity implementation provides this economic barrier, making it costly for attackers to create multiple identities.

E. COMPARISON WITH STATE-OF-THE-ART

Table 1 summarizes the key advantages of the proposed approach over existing methods. Unlike voting-based systems, our method captures pairwise behavioral patterns. Unlike static graph-based methods [7], our approach adapts to environmental conditions and provides a complete on-chain implementation. The combination of volatility-aware trust decay with economic incentive compatibility (slashing) creates a comprehensive defense against both strategic attacks and environmental disruptions.

VIII. CONCLUSION

In this paper, we identified and formalized the problem of Reputation Inertia in graph-based blockchain oracles, where accumulated history masks recent malicious behavior (Whitewashing) and environmental volatility triggers false positives (Flash Crashes).

We proposed a novel dynamic profiling mechanism based on an Adaptive Exponential Weighted Moving Average (AEWMA). By modulating the learning rate α_t via real-time network volatility, our system is hypersensitive to attacks during stability and robust to noise during volatility. We provided a complete Solidity smart contract implementation demonstrating the on-chain feasibility of the mechanism with $O(1)$ storage per edge weight. Through extensive Monte Carlo simulations (100 runs) using a 20-node oracle network with historical cryptocurrency price dynamics, we demonstrated that the proposed method reduces the average Time-to-Detection for whitewashing attacks from 9.0 rounds (static baseline) to 1.7 rounds, representing an 81% improvement. Simultaneously, the AEWMA mechanism maintained a False Positive Rate of only 0.4% under extreme volatility conditions ($5\times$ standard deviation), compared to 7.1% for the static approach. The flash crash stability analysis further demonstrated that AEWMA achieves self-healing trust recovery within 5 rounds of crash termination, while static methods suffer permanent trust degradation. Parameter sensitivity analysis confirmed that $\beta = 1.0$ provides the optimal trade-off between detection speed and false positive suppression across varying market conditions. Future work will focus on integrating this reputation model with Zero-Knowledge Proofs (ZKPs) to allow for fully private and scalable on-chain reputation verification, and extending the evaluation to larger-scale networks with heterogeneous adversarial strategies.

REFERENCES

- [1] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system.” White Paper, 2008.
- [2] V. Buterin, “Ethereum: A next-generation smart contract and decentralized application platform.” White Paper, 2014.
- [3] A. Egberts, “The oracle problem-an analysis of how blockchain oracles undermine the advantages of decentralized ledger systems,” SSRN Electronic Journal, 2017. doi: 10.2139/ssrn.3382343.
- [4] H. Al-Breiki, M. H. U. Rehman, K. Salah, and D. Svetinovic, “Trustworthy blockchain oracles: Review, comparison, and open research challenges,” IEEE Access, vol. 8, pp. 85675–85685, 2020.
- [5] A. Pasdar, Y. C. Lee, and Z. Dong, “Connect api with blockchain: A survey on blockchain oracle implementation,” ACM Computing Surveys, vol. 55, no. 10, pp. 1–39, 2023.
- [6] Y. Cai, N. Irtija, E. E. Tsiropoulou, and A. Veneris, “Truthful decentralized blockchain oracles,” International Journal of Network Management, vol. 32, no. 2, p. e2179, 2022.
- [7] K. Almi’ani, Y. C. Lee, T. Alrawashdeh, and A. Pasdar, “Graph-based profiling of blockchain oracles,” IEEE Access, vol. 11, pp. 24995–25007, 2023.
- [8] K. Hoffman, D. Zage, and C. Nita-Rotaru, “A survey of attack and defense techniques for reputation systems,” ACM Computing Surveys (CSUR), vol. 42, no. 1, pp. 1–31, 2009.
- [9] L. Breidenbach et al., “Chainlink 2.0: Next steps in the evolution of decentralized oracle networks.” Chainlink Labs White Paper, 2021.
- [10] K. Nelaturu, J. Adler, M. Merlini, R. Berryhill, N. Veira, Z. Poulos, and A. Veneris, “On public crowdsourced mechanisms for a decentralized blockchain oracle,” IEEE Transactions on Engineering Management, vol. 67, no. 4, pp. 1444–1458, 2020.
- [11] J. R. Douceur, “The sybil attack,” in International workshop on peer-to-peer systems, pp. 251–260, Springer, 2002.
- [12] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina, “The eigentrust algorithm for reputation management in p2p networks,” in Proceedings of the 12th international conference on World Wide Web, pp. 640–651, 2003.
- [13] Z. Yan, P. Zhang, and A. V. Vasilakos, “A survey on trust management for internet of things,” Journal of Network and Computer Applications, vol. 42, pp. 120–134, 2014.
- [14] Y. Liu, Y. Zhou, J. Wang, and T. Philbeck, “Measurement and analysis of the makerdao black thursday,” in Proceedings of the 2021 ACM CCS Workshop on Decentralized Finance and Security, pp. 1–5, 2021.
- [15] V. Buterin, D. Reijersbergen, S. Leonardos, and G. Piliouras, “Incentives in Ethereum’s hybrid Casper protocol,” in 2019 IEEE 27th International Conference on Network Protocols (ICNP), pp. 1–11, IEEE, 2019.
- [16] C. Li, B. Palanisamy, and W. Xu, “Gas estimation and optimization for smart contracts on ethereum,” in Proceedings of the 2021 IEEE International Conference on Web Services (ICWS), pp. 1–8, IEEE, 2021.
- [17] H. Al-Breiki, M. Rehman, K. Salah, and D. Svetinovic, “Blockchain oracles: State-of-the-art and research directions,” IEEE Access, vol. 8, pp. 20185–20202, 2020.

...